Queue using array

แล้วก็เป็นอีกครั้งคนออกโจทย์ ขี้เกียจแต่งเนื้อเรื่องเหมือนเดิมอีกแล้ว เอาเป็นว่าจะให้โจทย์แบบนี้ละกัน กำหนดให้มี Queue หนึ่งชุดที่เก็บข้อมูลด้วย **อาร์เรย์ขนาดคงที่ 30000 ช่อง** ตอนเริ่มต้น Queue ยังว่างอยู่ ระบบจะดำเนินการทั้งหมด n คำสั่ง แต่ละคำสั่งคือการเพิ่มข้อมูลเข้าท้ายแถว เอาข้อมูลออกจากหัวแถว ดูข้อมูลหัวแถว ดูข้อมูลท้ายแถว ถามขนาด หรือแสดงข้อมูลทั้งหมดใน Queue คุณต้องเขียนเฉพาะฟังก์ชันสำหรับจัดการ Queue เท่านั้น คำสั่งทั้ง n คำสั่งมีรูปแบบดังต่อไปนี้

คำสั่ง	การทำงาน
enqueue x	เพิ่มค่า x ไว้ท้ายสุดของ Queue
dequeue	เอาข้อมูลที่อยู่หัวแถวออกจาก Queue
front	ดูค่าที่หัวแถวโดยไม่เอาออก
rear	ดูค่าที่ท้ายแถวโดยไม่เอาออก
size	แสดงจำนวนข้อมูลใน Queue
display	แสดงข้อมูลทั้งหมดใน Queue จากหัวแถวไปท้ายแถว

คำสั่งต่าง ๆ อาจถูกเรียกตอนที่ Queue ว่างหรือเต็มก็ได้ โปรแกรมของคุณต้องจัดการกรณีเหล่านี้ให้ถูกต้อง

งานของคุณ (Your Task)

เขียนฟังก์ชันสำหรับจัดการ Queue ที่ใช้อาร์เรย์ให้ทำงานถูกต้องตามคำสั่งที่กำหนด โดยต้องเขียนฟังก์ชันสำหรับ

- ตรวจสอบว่า Queue ว่างและเต็มหรือไม่
- เพิ่มข้อมูล (enqueue)
- เอาข้อมูลออก (dequeue)
- ดูข้อมูลหัวแถว (front)
- ดูข้อมูลท้ายแถว (rear)
- นับจำนวนข้อมูล (size)
- แสดงข้อมูล (display)

ข้อนี้มีรูปแบบการเขียนโปรแกรมที่แตกต่างจากรูปแบบปรกติ

โปรดดูส่วนท้ายของโจทย์ข้อนี้เกี่ยวกับวิธีการใช้งาน

รายละเอียดของโปรแกรม

คุณต้องเขียนฟังก์ชันต่อไปนี้

```
bool isEmpty()
```

คืน true ถ้า Queue ไม่มีข้อมูล มิฉะนั้นคืน false

```
bool isFull()
```

คืน true ถ้าท้ายแถวอยู่ที่ช่องสุดท้ายของอาร์เรย์ (ช่องที่ 29999) ซึ่งหมายความว่าไม่สามารถ enqueue เพิ่มได้อีก มิฉะนั้นคืน false

```
void enqueue(int value)
```

value คือค่าที่จะเพิ่มไว้ท้ายสุดของ Queue

ถ้า Queue เต็ม ให้พิมพ์ Queue Overflow ขึ้นบรรทัดใหม่ และ ห้ามเปลี่ยนแปลงข้อมูลใน Queue

```
void dequeue()
```

เอาข้อมูลที่อยู่หัวแถวออกจาก Queue

ถ้า Queue ว่าง ให้พิมพ์ Queue Underflow ขึ้นบรรทัดใหม่ และ ห้ามเปลี่ยนแปลงสถานะของ Queue

```
int front()
```

คืนค่าที่หัวแถว โดย ไม่เอาค่านั้นออก

ถ้า Queue ว่าง ให้พิมพ์ Queue is empty ขึ้นบรรทัดใหม่ แล้วคืนค่า -1

```
int rear()
```

คืนค่าที่ท้ายแถว โดย ไม่เอาค่านั้นออก

ถ้า Queue ว่าง ให้พิมพ์ Queue is empty ขึ้นบรรทัดใหม่ แล้วคืนค่า -1

```
int size()
```

คืนจำนวนข้อมูลที่อยู่ใน Queue ขณะนั้น

```
void display()
```

แสดงข้อมูลทั้งหมดตั้งแต่หัวแถวไปท้ายแถว แต่ละค่าตามด้วยช่องว่าง 1 ตัว แล้วขึ้นบรรทัดใหม่

ถ้า Queue ว่าง ให้พิมพ์ Queue is empty ขึ้นบรรทัดใหม่

ข้อควรระวัง: ข้อความ Queue Overflow, Queue Underflow, Queue is empty และการแสดงผลของ display ต้องพิมพ์ในฟังก์ชันของคุณเอง ส่วนค่าที่ front, rear และ size คืนมา grader จะเป็นผู้พิมพ์ให้ ดังนั้น เมื่อ front หรือ rear ตอนว่าง ผลลัพธ์จะมี 2 บรรทัด คือ Queue is empty จากฟังก์ชันของคุณ และ -1 จาก grader

ข้อควรระวังเพิ่มเติม: เนื่องจากช่องที่เอาข้อมูลออกไปแล้วไม่ถูกนำกลับมาใช้ ตลอดการทำงานทั้งหมดจะ enqueue สำเร็จได้รวมกันไม่เกิน 30000 ครั้ง ไม่ว่าจะมีการ dequeue ไปแล้วกี่ตัวก็ตาม ตัวอย่างเช่น ถ้า enqueue ครบ 30000 ตัวแล้ว dequeue ออก 1 ตัว Queue จะมีข้อมูล 29999 ตัว (size คืน 29999) แต่ enqueue ต่อไม่ได้ ต้องพิมพ์ Queue Overflow นอกจากนี้ Queue อาจ ว่างและเต็มพร้อมกัน ได้ เช่น enqueue 30000 ตัวแล้ว dequeue ออกหมด isEmpty() จะคืน true และ isFull() จะคืน true ด้วย

ขอบเขตข้อมูล

- $1 <= n <= 100000$ โดย n คือจำนวนคำสั่งทั้งหมด
- $-2^{31} <= value <= 2^{31} - 1$
- อาร์เรย์มีขนาด 30000 ช่อง และ Queue เป็น Queue ว่างก่อนเริ่มคำสั่งแรก
- คำสั่งที่ 1 ถึง n เป็นหนึ่งในคำสั่ง enqueue, dequeue, front, rear, size หรือ display ตามที่อธิบายไว้
- ห้ามใช้ class, pointer และ container ของ STL (เช่น vector, queue) ในการเก็บข้อมูล ให้ใช้อาร์เรย์และตัวแปรจำนวนเต็มเท่านั้น

ตัวอย่าง

Input	Output
5 dequeue front enqueue 5 display size	Queue Underflow Queue is empty -1 5 1

8 enqueue 10 enqueue 20 enqueue 30 display front rear dequeue size	10 20 30 10 30 2

ข้อกำหนด

ระยะเวลาสูงสุดที่ใช้ในการประมวลผล	1 วินาที
หน่วยความจำสูงสุดที่ใช้ในการประมวลผล	16 MB
คะแนนสูงสุดของโจทย์	100 คะแนน

คำแนะนำในการเขียนโปรแกรม

สำหรับข้อนี้ วิธีการเขียนโปรแกรมและส่งโปรแกรมเข้าสู่ระบบเกรดเดอร์ (Grader) จะเป็นรูปแบบที่แตกต่างจากรูปแบบทั่วไปของการเขียนโปรแกรม ที่คุณจะต้องเขียนโปรแกรมในส่วนรับข้อมูลนำเข้าและส่วนแสดงผล โดยในข้อนี้คุณจะต้องเขียนโปรแกรมเฉพาะส่วนการคำนวณที่จำเป็นลงในฟังก์ชันที่กำหนดให้ โดยข้อมูลนำเข้าต่าง ๆ จะสามารถใช้ได้ผ่านพารามิเตอร์ของฟังก์ชัน และผลการคำนวณหรือการเปลี่ยนแปลงของ Queue จะดำเนินการผ่านฟังก์ชันดังกล่าว โดยที่คุณไม่ต้องเขียนโปรแกรมในส่วนรับข้อมูลนำเข้า และไม่ต้องเขียนส่วนอ่านคำสั่ง ยกเว้นข้อความที่ระบุให้พิมพ์ไว้ในรายละเอียดของแต่ละฟังก์ชัน รูปแบบส่วนนี้ยึดตามคำแนะนำของโจทย์ตัวอย่าง

คุณจะได้รับไฟล์ต่างๆ 7 ไฟล์ดังนี้ คุณจะต้องแก้ไขไฟล์เดียวตามที่โจทย์กำหนด และส่งไฟล์ดังกล่าวเข้าสู่ระบบ

- grader.cpp เป็นไฟล์หลักที่ทำหน้าที่รับข้อมูลนำเข้า เรียกใช้ฟังก์ชัน และพิมพ์ค่าที่ front(), rear() และ size() คืนมา
- C1T47.h เป็นไฟล์ส่วนหัวซึ่งประกาศฟังก์ชันต่าง ๆ ที่ใช้ในโจทย์

- C1T47.cpp เป็นไฟล์ที่คุณจะต้องเขียนฟังก์ชันทั้ง 8 ฟังก์ชัน และเป็นไฟล์ที่คุณส่งเข้าสู่ระบบ (ในไฟล์ C1T47.cpp **ห้ามมีฟังก์ชัน main()** แต่มีตัวแปรหรือฟังก์ชันอื่น ๆ ได้)
- run_cpp.sh เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการรันโปรแกรมทั้งหมดได้บนระบบ Linux และ macOS
- compile_cpp.sh เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการคอมไพล์โปรแกรมได้บนระบบ Linux และ macOS
- run_cpp.bat เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการรันโปรแกรมทั้งหมดได้บนระบบ Windows
- compile_cpp.bat เป็นไฟล์ที่คุณสามารถเรียกใช้เพื่อทำการคอมไพล์โปรแกรมได้บนระบบ Windows

คุณจะได้รับไฟล์ .zip ซึ่งสามารถดาวน์โหลดได้จากระบบ โดยในไฟล์ดังกล่าวเมื่อขยาย (extract) ออกมาแล้วจะมีไฟล์ทั้งหมดอยู่ คุณควรใช้ text editor หรือ IDE ในการแก้ไขไฟล์ C1T47.cpp และทำการคอมไพล์หรือรันโปรแกรมผ่านการเรียกใช้ compile_cpp.sh หรือ run_cpp.sh หรือ compile_cpp.bat หรือ run_cpp.bat

เมื่อคุณเรียกไฟล์ run_cpp.sh หรือ run_cpp.bat เพื่อทำการรันโปรแกรม โปรแกรมจะทำงานตามที่ได้ระบุไว้ในไฟล์ grader.cpp ซึ่งจะเป็นผู้รับคำสั่ง และเรียกใช้ฟังก์ชันใน C1T47.cpp ตามคำสั่งที่ได้รับ รูปแบบการแยก grader ออกจากไฟล์ที่คุณส่งเป็นไปตามแนวทางของโจทย์ตัวอย่าง